



ChurnGuard

Predict SaaS customer churn before they leave

Part 2 of 2 — **Codebase & File Guide**

What every important file does, and how the folders fit together.

Prepared 14 July 2026 · Author: Bhanu Prakash

o • The folder map

A single Next.js app — the API route handlers are the backend. Nothing touches the database or AI except code in `src/lib/`.

```
ChurnGuard/ (churnguard)
├─ prisma/
│  ├─ schema.prisma      # tables + enums (PostgreSQL)
│  ├─ seed.ts            # demo operator + 8 customer personas + history
│  └─ score-all.ts      # CLI: re-score the whole book
├─ src/
│  ├─ app/                # routes (folder = URL)
│  │  ├─ dashboard/ customers/[id]/
│  │  ├─ login/ page.tsx layout.tsx globals.css
│  │  └─ api/             # route handlers = backend
│  ├─ lib/                # the brain: db, auth, scoring, persistence
│  ├─ components/        # Navbar, charts, per-page views
│  └─ types/
├─ .env                  # DB url, auth secret, AI keys (no .env.example)
└─ package.json / next.config.ts / tsconfig.json
```

1 • Data layer (prisma/)

DATABASE BLUEPRINT

`prisma/schema.prisma`

Declares the PostgreSQL datasource and every table. Models: `User` (bcrypt operator auth); `Customer` (company, plan, MRR, status); `ActivityEvent` (the raw usage stream); `RiskScore` (current score + band + reason + action + `model` engine tag, one per customer); `RiskScoreHistory` (append-only weekly snapshots powering the trend chart). Enums: `Plan`, `CustomerStatus`, `ActivityType`, `RiskBand` (HIGH/MEDIUM/LOW).

DATA GENERATOR

`prisma/seed.ts`

Seeds the demo operator plus **8 synthetic customer personas**, each with a hand-built `ActivityEvent` history (logins, feature uses, tickets, failed payments, NPS) engineered to land across the risk bands, and **6 weekly synthetic `RiskScoreHistory` points** per customer — so the dashboard, the donut, the revenue-by-band bars and the trend charts all have a real narrative on first boot.

BATCH SCORER CLI

`prisma/score-all.ts`

A command-line script (`npm run db:score`) that runs the scorer over every customer and persists the results — the same work the dashboard's "Re-score all" button triggers, runnable from the terminal.

2 • The brain (src/lib/)

THE AI SCORER ★

`src/lib/scoring.ts`

The heart of the app: `extractFeatures()` (raw events → deterministic churn signals), the transparent additive **heuristic**, the **Claude** forced tool-use call (`report_churn_risk`), the **Gemini** REST call (`responseSchema`), and `scoreCustomer()` — the Claude → Gemini → heuristic cascade returning the first tier that succeeds. Both LLM tiers share one identical prompt.

PERSISTENCE

`src/lib/score-service.ts`

`scoreAndSave()` : runs `scoreCustomer()` , then in a single `prisma.$transaction` upserts the current `RiskScore` and appends a `RiskScoreHistory` snapshot — the two writes that keep the score and the trend chart consistent.

DATABASE CLIENT

`src/lib/prisma.ts`

One shared `PrismaClient` (global singleton) so dev hot-reload doesn't exhaust the connection pool.

AUTH + UTILS

`src/lib/auth.ts · session.ts · utils.ts`

NextAuth config (Credentials + JWT + bcrypt); `getSession()` server helper; formatters — band colours, `formatMoney`, and `timeAgo` relative timestamps.

3 · Routes, API & components

FILE	PURPOSE
app/page.tsx · login/ · layout.tsx · globals.css	Landing page, combined login/register, root layout + global styles.
app/dashboard · customers/[id] /page.tsx	Server components; guard the session, read via lib/ , hand JSON to a client view (the churn-overview dashboard and the per-customer detail page).
app/api/customers · customers/[id]	GET list / POST create · GET / PATCH / DELETE one customer.
app/api/customers/[id]/score	POST re-score a single customer (runs scoreAndSave).
app/api/customers/[id]/events	POST ingest an ActivityEvent — the real-world webhook path (and the "Log activity" form's endpoint).
app/api/score-all	POST re-score the whole book (the dashboard "Re-score all" button).
app/api/register · auth/[...nextauth]	Create operator account · NextAuth handler.
components/Navbar · Providers · RiskBadge	App chrome (nav bar), NextAuth SessionProvider, the colour-coded band badge.
components/dashboard/StatCard · RiskCharts · CustomerTable	The four stat cards; RiskCharts = RiskDonut + RevenueAtRiskChart ; the AI-annotated, searchable, filterable customer table.
components/customer/CustomerActions · LogEventForm · RiskTrend	Per-customer re-score/edit/delete actions; the log-activity form; the risk-trend line chart.
types/next-auth.d.ts	Session/JWT type augmentation for NextAuth.

4 · Commands & the honest caveats

COMMAND	DOES
npx prisma db push	Create DB + tables from the schema (no migrations folder)
npm run db:seed	Seed the demo operator + 8 customer personas + history
npm run db:score	Re-score every customer from the CLI
npm run dev	Dev server → localhost:3000 · demo@churnguard.app / demo1234

Honesty notes for the interview (code is ground truth): the README and tech-stack blurb say **MySQL 9 + Claude only**, but the real schema.prisma and .env use **PostgreSQL**, and the code ships an **undocumented Google Gemini fallback tier** (the scorer cascade is Claude → Gemini → heuristic). The committed migration SQL is **MySQL-dialect** — stale versus the actual Postgres schema. There is **no**

`.env.example`, even though the README tells you to copy one. And the committed `.env` holds **real secrets** that would normally never be committed.